

Information Retrieval 9

Query and Document Expansion

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

monoT5: Ranking via the Seq2Seq framework. Generates "true"/"false" from input text and estimates relevance from the probabilities

duoT5: Determines relative relevance of document pairs through Pairwise comparison. Improves accuracy but increases computational cost

RankGPT: Listwise reranking by LLMs. A new paradigm that directly reorders candidate document lists

This time, as an approach different from reranking, we learn query expansion and document expansion methods that improve **first-stage retrieval** itself

We learn a family of methods that approach the fundamental limitation of keyword search -- the **vocabulary mismatch problem** -- from both the query side and the document side

First Half: The Problem and Basic Approaches

Revisiting the Vocabulary Mismatch Problem

Query Expansion (RM3, Neural Query Expansion)

doc2query / doc2query--

Second Half: Neural Document Expansion and Operations

DeepCT (Contextual Term Weighting)

DeepImpact (Integrating Expansion + Weighting)

Query Expansion vs Document Expansion

Practical Considerations

The Vocabulary Mismatch Problem

The Vocabulary Mismatch Problem

Vocabulary Mismatch is the problem where a user's query and relevant documents express the same concept using **different vocabulary**

Concrete examples:

Query: "car fuel efficiency" -> Document:
"automobile **miles per gallon**"

Query: "laptop battery life" -> Document:
"notebook **power duration**"

Query: "heart attack prevention" -> Document:
"**myocardial infarction** risk reduction"

In keyword-based methods like BM25, if the query and document share **no common terms**, the score becomes 0

Query: "car fuel efficiency"

Document A: "How to improve **car fuel efficiency**"

-> BM25: **High score** (terms match)

Query: "car fuel efficiency"

Document B: "Technology to reduce **automobile gasoline consumption** per mile"

-> BM25: **Low score** (terms do not match)

Document B is clearly relevant, but BM25 cannot rank it appropriately

Approaches to resolving vocabulary mismatch can be broadly classified into two categories

1. Query Expansion

Adds related terms to the **query side** to broaden the search scope

Online processing: Executed at search time

Example: "car fuel efficiency" -> "car **automobile** fuel efficiency **gasoline consumption**"

2. Document Expansion

Adds related terms to the **document side** to make documents more discoverable

Offline processing: Executed at indexing time

Example: Adding anticipated query terms to documents

Query Expansion

Query q → Expansion → $q + \text{added terms}$

Executed for each search (online)

Document Expansion

Document d → Expansion → $d + \text{added terms}$

Executed once in advance (offline)

Lin et al. (2021) describe query expansion and document expansion as "**yin and yang**": two complementary approaches

Query Expansion

Query Expansion

Pseudo-Relevance Feedback (PRF) is one of the most classical query expansion methods

Basic idea: Assume the top-ranked documents from the initial search are "pseudo-relevant" and extract expansion terms from them

RM3 (Relevance Model 3) [Lavrenko & Croft, 2001]:

1. Execute BM25 retrieval with query q
2. Treat the top k documents as the **pseudo-relevant document set $\mathcal{F}$**
3. Estimate term occurrence probabilities within $\mathcal{F}$ and construct an expanded query through **linear interpolation** with the original query

$$P_{\text{RM3}}(w|q) = \alpha \cdot P_{\text{ML}}(w|q) + (1 - \alpha) \cdot P_{\text{RM}}(w|\mathcal{F})$$

α : Hyperparameter controlling the balance between original query and expansion terms

RM3 Processing Flow

Original query: "Kyoto autumn leaves best time"

▼ Initial retrieval with BM25

Extract expansion terms from top documents

"Arashiyama" "Tofukuji" "November" "illumination" ...

▼ Integrate via linear interpolation

Expanded query: "Kyoto autumn leaves best time Arashiyama Tofukuji November ..."

▼ Re-search with expanded query

Final results: Retrieve more relevant documents

To overcome PRF's limitations, **neural query expansion** has been studied

BERT-QE [Zheng et al., 2020]: uses BERT's contextual understanding to select expansion terms from pseudo-relevant documents

Neural PRF [Li et al., 2018]: learns term importance from pseudo-relevant documents with neural networks

However, neural query expansion still faces:

Model inference at search time → **increased latency**

Short queries (2-5 words) make expansion unstable; dependence on initial retrieval remains

Evolution of Query Expansion

Classical methods

Expansion using **synonym dictionaries** (WordNet, etc.)
-> Limited by dictionary coverage

Statistical methods

RM3: Probabilistic expansion term estimation from pseudo-relevant documents
-> Query drift problem

Neural methods

BERT-QE, etc.: Expansion term selection based on contextual understanding
-> Latency and initial retrieval dependency

A shift in thinking: If we expand the **document side** instead of the query, it can be preprocessed offline without affecting search-time latency

doc2query

Document Expansion by Query Prediction

doc2query [Nogueira et al., 2019] generates "queries for which this document would be the answer" and appends them to the document

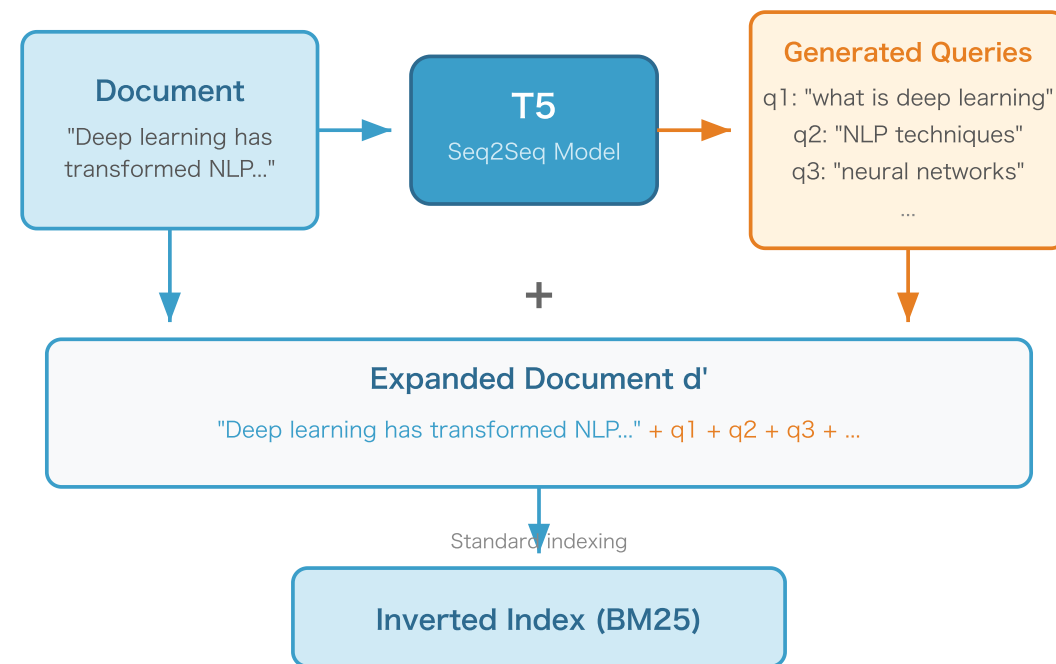
Basic Idea:

A Seq2Seq model generates queries from a document, then **concatenates** them to the original text for indexing

At search time, use standard BM25 (no modification)

Why this is effective: query terms users would actually type — including **synonyms/paraphrases** not in the document — are added, mitigating BM25's vocabulary mismatch

Conceptual Diagram of doc2query



Document expansion flow of doc2query (based on Lin et al. (2021), Figure 19)

doc2query adds "**question**" perspectives not explicitly written in the document (see example, right)

Results on MS MARCO Passage Ranking:

Method	MRR@10
BM25	0.184
BM25 + RM3	0.156
BM25 + doc2query	0.218

How doc2query Resolves Mismatch

Original document

"Osaka Castle was built by Toyotomi Hideyoshi in 1583. The observation deck offers a panoramic view of Osaka city..."

▼ doc2query

Generated queries

"Osaka Castle **who built it**"

"Osaka Castle **observation** view"

"**Hideyoshi's castle** history"

Effect

Expressions **not in the original document** such as "who built it," "observation," and "Hideyoshi's castle" are added, enabling matching with a wider variety of queries

Method: Nogueira et al., 2019. Numbers: Lin et al., "Pretrained Transformers for Text Ranking", 2021, Table 34 (MS MARCO passage, dev MRR@10)

doc2query-- (often called docTTTTTquery) [Nogueira and Lin, 2019] is an improved version of doc2query that strengthens document expansion primarily through **T5-based generation** and **large-scale query generation**

Key improvements over doc2query:

1. **T5 model**: switched from Transformer Seq2Seq to **T5** → higher-quality generation
2. **Large-scale: 40-80 queries** per document (doc2query: 5-10)
3. **Diverse** query candidates via sampling

Also called *docTTTTTquery* (used interchangeably)

doc2query vs doc2query--

doc2query (original version)

Model: Transformer Seq2Seq
Generation count: 5-10 queries / document
Filtering: None

doc2query-- (improved version)

Model: **T5** (more powerful pre-trained model)
Generation count: **40-80** queries / document
Filtering: Depends on implementation and experimental settings (varies across methods)

T5's powerful text generation capability enables the generation of more natural and diverse queries

DeepCT

Deep Contextualized Term Weighting

In BM25, term importance is mainly determined by **TF (Term Frequency)** and **IDF**

TF ignores context: importance comes from raw occurrence count, regardless of whether a term is **on-topic** — a generic word like "recent" can outweigh a topical term (see example, right)

Goal of DeepCT: use BERT to learn each term's **contextual importance**, replacing raw TF

TF vs Contextual Importance

TF-based weights (conventional)

"Recent deep learning for image recognition uses the recent methods on the recent datasets..."

TF: recent=3, deep learning=1, image recognition=1

Contextual importance (DeepCT)

For the same document:

deep learning=0.85, image recognition=0.78, recent=0.12

-> Assigns high weights to terms related to the document's main topic

DeepCT uses BERT's contextual understanding to identify **"terms that are truly important to the document's main topic"**

DeepCT [Dai and Callan, 2019] uses BERT to predict a context-dependent importance weight for each term, replacing raw TF

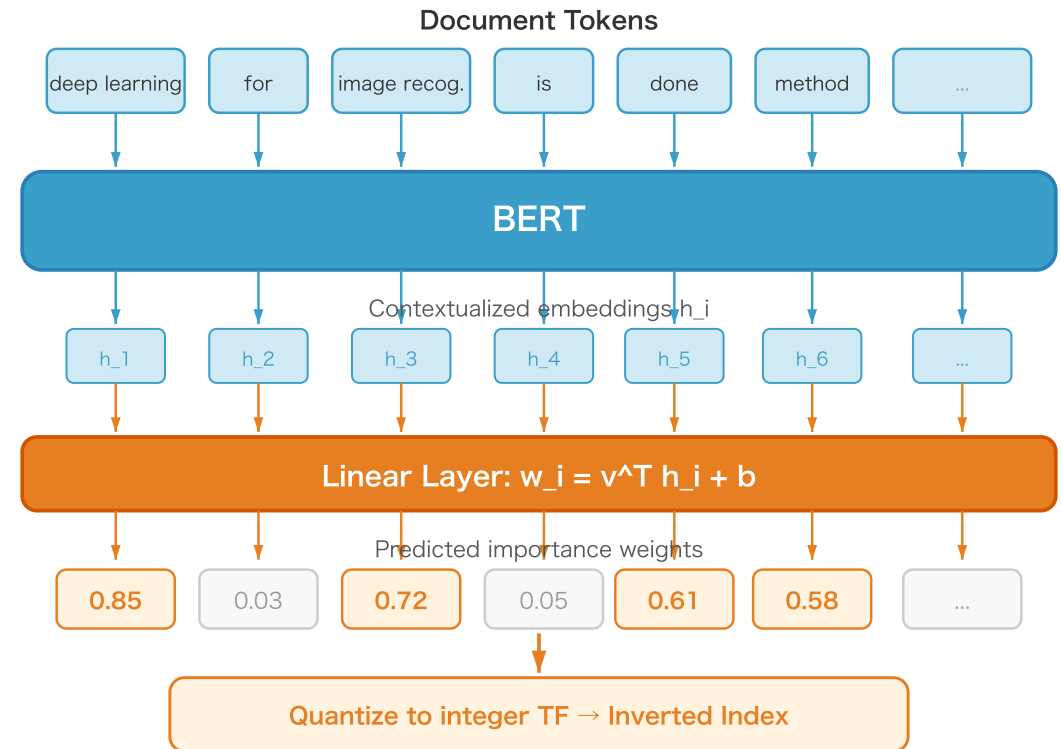
How it works (see figure):

BERT output vector $\mathbf{h}_i$ per token $\rightarrow$ linear layer

$$w_i = \mathbf{v}^T \mathbf{h}_i + b$$

Weights quantized to integers and stored as **pseudo-TF** in the inverted index $\rightarrow$ no extra cost at search time

Training: MS MARCO (query, relevant doc) pairs;
query terms $\rightarrow$ target weight 1, others $\rightarrow$ 0; MSE regression loss



Processing is needed to use the real-valued weights output by DeepCT directly in the inverted index

Quantization Process:

1. DeepCT predicts each token's weight $w_i \in [0, 1]$
2. Scale the weights: $\hat{w}_i = \text{round}(w_i \times N)$
 N : Scaling factor (e.g., $N = 100$)
3. Use $\hat{w}_i$ as **pseudo TF**
4. Store in the inverted index: Record pseudo TF in the posting list

Integration with BM25:

Replace the original TF with DeepCT's pseudo TF
IDF is used as-is

No changes needed to the BM25 formula -- only the TF values change

Quantization and Indexing Flow

DeepCT output: {deep learning: 0.85, image: 0.72, of: 0.03, method: 0.61}

▼ Scaling ($N=100$)

Quantization: {deep learning: 85, image: 72, of: 3, method: 61}

▼ Store as pseudo TF

Inverted index:

"deep learning" -> (doc1, tf=85), ...
"image" -> (doc1, tf=72), ...

A major advantage of DeepCT is that existing BM25 infrastructure can be used as-is

Results on MS MARCO Passage Ranking [Dai and Callan, 2019]:

Method	MRR@10
BM25 (TF-based)	0.184
BM25 + RM3	0.156
DeepCT + BM25	0.243

Characteristics of DeepCT:

- Index size unchanged:** only replaces TF values, adds no terms
- Fast search:** plain BM25 at query time, no extra cost
- Limitation:** no weights for absent terms → **no vocabulary expansion**

Characteristics Comparison: doc2query vs DeepCT

	doc2query	DeepCT
Method	Add terms	Adjust term weights
Vocabulary expansion	Yes	No
Weight adjustment	No	Yes
Index	Inflates 2-3x	No change
Search speed	Slightly slower	No change

doc2query **expands vocabulary** but cannot control weights. DeepCT **optimizes weights** but cannot expand vocabulary. The two are **complementary**

Method: Dai and Callan, 2019. Numbers: Lin et al., 2021, Table 34 (MS MARCO passage, dev MRR@10)

DeepImpact

Combining Expansion and Weighting

doc2query and DeepCT addressed different aspects: **vocabulary expansion** and **term weighting** respectively

DeepImpact [Mallia et al., 2021] is a method that **integrates** these two

Basic Idea (see pipeline, right): **doc2query--expansion** + neural **Impact Score** weighting (an extension of DeepCT), stored as pseudo-TF in the inverted index

Differences from DeepCT:

Weights **expanded terms too** (not just original document terms)

Optimized directly for relevance (vs DeepCT's regression task)

DeepImpact Pipeline

Original document d

▼ Expand with doc2query--

Expanded document $d' = d + \text{generated queries}$

▼ Predict per-token weights with BERT

Impact Score calculation

For each unique term t , learn $s(t, d')$

▼ Quantize and store

Inverted index

$t \rightarrow (\text{doc1}, \text{impact}=42), (\text{doc2}, \text{impact}=18), \dots$

DeepImpact is a pipeline that integrates **document expansion (doc2query--)** and **neural weight learning (BERT)**

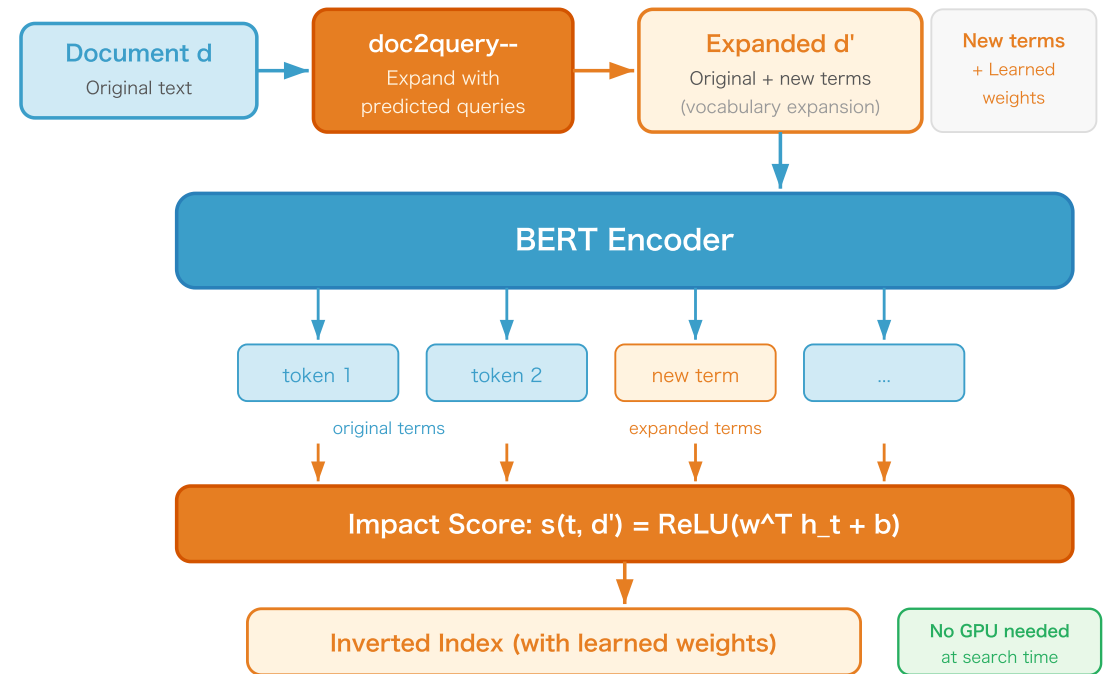
Processing flow:

1. Expand the original document d with doc2query-- for **vocabulary expansion**
2. Input the expanded document d' to BERT and obtain contextualized representations for each token
3. Predict an **Impact Score** for each token
4. Quantize and store in the inverted index

Key points:

Both original terms (blue) and expanded new terms (orange) receive learned weights

At search time, standard sparse retrieval using the inverted index is used -> **No GPU required**



Comparison on MS MARCO Passage Ranking [Mallia et al., 2021]:

Method	MRR@10
BM25	0.184
doc2query-- + BM25	0.277
DeepCT + BM25	0.243
DeepImpact	0.326

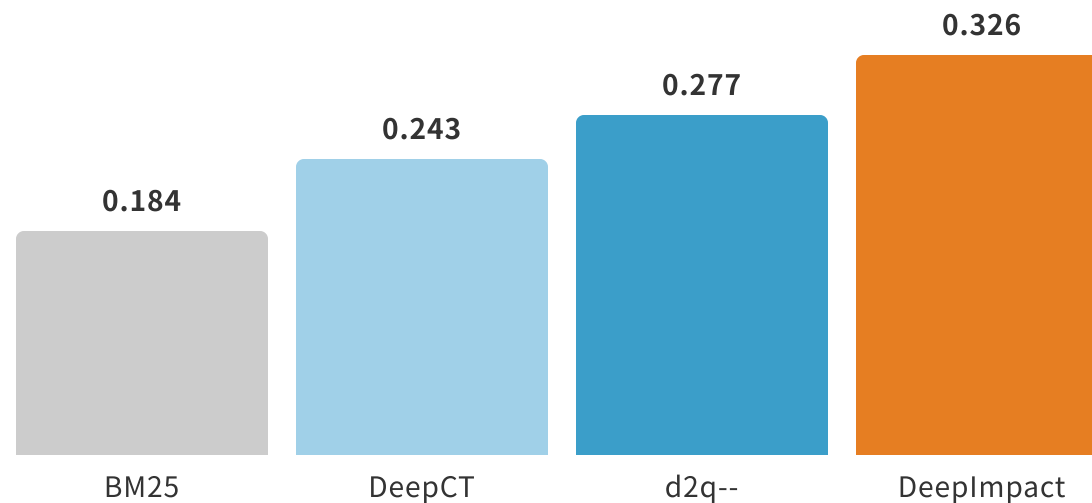
DeepImpact shows a **+77%** improvement over BM25

Advantages of DeepImpact:

Achieves **vocabulary expansion** and **weight optimization** simultaneously

Inverted index-based, so **search is fast**
(comparable to BM25)

MRR@10 Comparison (MS MARCO Passage)



Numbers: Lin et al., 2021, Table 36 (dev MRR@10)

The **integration** of vocabulary expansion (doc2query--) + weight learning (an extension of DeepCT) brought significant improvements

Query Expansion vs Document Expansion

Comparing the Two Approaches

The two approaches are **complementary**, each with advantages and disadvantages

Advantages of query expansion:

- No need to re-index the entire document collection
- Advantageous when the collection is frequently updated
- Enables flexible expansion tailored to query intent

Advantages of document expansion:

- Does not affect search-time latency
- Can leverage the rich context of documents
- Once processed, effective for all queries

Combination is also possible: One can apply RM3 for query expansion on an index already expanded with doc2query--

Detailed Comparison

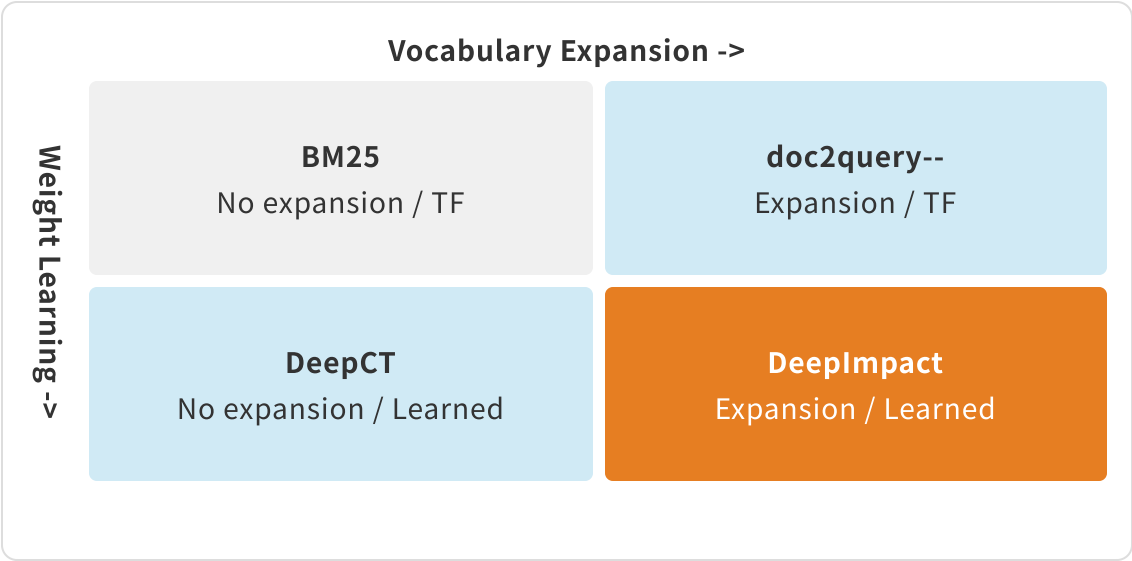
Aspect	Query Expansion	Document Expansion
Processing timing	Online	Offline
Latency impact	Yes	None
Index size	No change	Increases
Collection updates	Easy	Reprocessing needed
Information source	Query + search results	Entire document
Scope of application	Per-query individual	Common to all queries
Representative methods	RM3, BERT-QE	doc2query, DeepCT

An overview of all methods covered in this section

Method	Vocab. Expansion	Weight Learning	MRR@10
BM25	-	-	0.184
+ RM3	(Query side)	-	0.156
+ doc2query	Document side	-	0.218
+ doc2query- -	Document side	-	0.277
DeepCT	-	Document side	0.243
DeepImpact	Document side	Document side	0.326

Trends: vocabulary expansion (*what to add*) and weight learning (*how to weight*) are **orthogonal** — DeepImpact combines both

Design Space of Methods



Numbers: Lin et al., 2021, Tables 34 & 36 (MS MARCO passage, dev MRR@10)

1. **The vocabulary mismatch problem** is a fundamental challenge that significantly impairs first-stage retrieval Recall. Two approaches exist: query expansion (online) and document expansion (offline)
2. **doc2query / doc2query--** are document expansion methods that generate queries from documents and append them. T5-based doc2query-- improves BM25 by +48% in MRR@10
3. **DeepCT** uses BERT to learn context-dependent term weights and replaces TF. Improves search quality without increasing index size
4. **DeepImpact** integrates vocabulary expansion and weight learning, improving BM25 by +77% in MRR@10. A pioneering method of Learned Sparse Retrieval

Next Lecture Preview: Bi-encoder and Dense Retrieval

The Bi-encoder vs Cross-encoder paradigm

Sentence-BERT, DPR, ANCE

ColBERT (Late Interaction)